

MANUAL TÉCNICO

KN-Store

Documento técnico de referencia para desarrolladores, administradores de infraestructura y equipos de mantenimiento de la plataforma de e-commerce KN-Store.

Joseph Varón – Nicolas Gil – Laura González – Santiago Flórez

Tabla de contenido

Tabla de contenido	2
1. Visión general del sistema.....	4
Características principales	4
2. Stack tecnológico	4
3. Arquitectura	5
3.1 Backend — Spring Boot	5
Patrones y principios	5
3.2 Frontend — React + TypeScript	5
Características del frontend	6
3.3 Modelo de dominio destacado	6
Entidades principales	6
Relaciones clave	6
4. Roles y seguridad	6
4.1 Roles definidos	6
4.2 Permisos por ruta (frontend).....	7
4.3 Ownership de recursos	7
4.4 Consideraciones de seguridad por diseño	7
5. Requisitos previos	7
5.1 Software necesario	7
5.2 Cuentas / servicios externos (opcionales)	8
6. Instalación y configuración.....	8
6.1 Clonar el repositorio	8
6.2 Instalar dependencias del frontend	8
6.3 Archivos de configuración principales	8
7. Base de datos	8
7.1 Modos de ejecución de MongoDB	8
7.2 Levantar MongoDB en desarrollo (recomendado).....	9
7.3 Seed de catálogo demo	9
7.4 Índices	9
7.5 Transacciones	9
8. API REST	9
8.1 Convenciones	9
8.2 Endpoints principales por dominio	9
8.3 Checkout	10
9. Frontend	10

9.1 Inicio del entorno de desarrollo	10
9.2 Build de producción.....	10
9.3 Consejos de desarrollo	11
10. Testing.....	11
10.1 Backend.....	11
10.2 Frontend.....	11
10.3 JMeter	11
11. Despliegue.....	11
11.1 Docker local completo	11
11.2 Imagen Docker con Jib.....	12
11.3 Empaquetado JAR	12
11.4 Consideraciones de producción	12
12. Utilidades y servicios clave	12
12.1 Servicios destacados del backend.....	12
12.2 Servicios destacados del frontend	12
13. Convenciones de desarrollo	13
13.1 Commits (Conventional Commits)	13
13.2 Flujo de trabajo Git.....	13
14. Archivos clave de referencia.....	13
15. Solución de problemas comunes.....	14
15.1 El checkout falla por transacciones.....	14
15.2 El frontend no encuentra el backend.....	14
15.4 Seed de catálogo no aparece	14

1. Visión general del sistema

KN-Store es una plataforma de comercio electrónico completa desarrollada para la tienda KN_STORE. Su propósito es migrar la operación manual —pedidos por WhatsApp, inventarios en hojas de cálculo— hacia un ecosistema digital integrado, seguro y escalable.

El sistema soporta catálogo público, carrito de compras, checkout atómico, pagos con pasarela simulada, facturación con PDF y QR, envíos, notificaciones por correo y un panel administrativo para gestión de inventario, usuarios y pedidos.

Características principales

- Tienda pública responsive (360 px – 1920 px).
- Panel administrativo con roles diferenciados.
- Carrito híbrido: localStorage para usuarios anónimos, backend para autenticados.
- Checkout atómico con transacciones MongoDB.
- Precios resueltos siempre del lado del servidor.
- Facturación con consecutivos diarios atómicos.
- Auditoría de transiciones de estado de pedidos y pagos.

2. Stack tecnológico

Capa	Tecnología	Versión / Notas
Backend	Spring Boot	4.0.6
Lenguaje backend	Java	21
Seguridad	Spring Security + JWT (OAuth2 Resource Server)	Tokens de 30 días
Base de datos	MongoDB	Replica set para transacciones
Migraciones DB	Mongock	Migraciones Java en config/dbmigrations
Frontend	React + TypeScript	React 19
Estado global	Redux Toolkit	—
Enrutamiento	React Router 7	—
UI	Bootstrap 5 / React-Bootstrap	Tema Bootswatch Cyborg
Bundler / tests	Webpack 5 / Vitest	—
Build backend	Maven	3.2.5+
Runtime Node	Node.js	24.16.0 LTS
Empaquetado	Jib	Imagen Docker / JAR ejecutable
Correo	SMTP	Configurable por perfil

3. Arquitectura

3.1 Backend — Spring Boot

Estructura de paquetes (src/main/java/com/mycompany/knstore/):

```

config/                # Configuración de Spring Boot
├─ dbmigrations/       # Migraciones Mongock
domain/                # Entidades MongoDB (@Document)
domain/enumeration/    # Enums de negocio
repository/            # Repositorios Spring Data MongoDB
service/               # Interfaces de servicio
service/impl/          # Implementaciones de servicios
service/dto/           # DTOs generados por MapStruct
service/mapper/        # Mapeadores MapStruct
service/util/          # Utilidades (MoneyUtils, MongoIdUtils, etc.)
service/payment/       # Gateway de pagos abstracto + simulado
service/invoice/       # Generación de facturas PDF
web/rest/              # Controladores REST
web/rest/errors/       # Manejo centralizado de errores
web/filter/            # Filtros (SPA, token inválido)
security/              # Utilidades JWT y roles
aop/logging/           # Aspectos de logging

```

Patrones y principios

- API REST stateless bajo /api/**.
- Patrón DTO + MapStruct para desacoplar la API del modelo de persistencia.
- Repository pattern con Spring Data MongoDB.
- Seguridad declarativa con @PreAuthorize.
- Lógica de negocio encapsulada en servicios; controladores delegan.

3.2 Frontend — React + TypeScript

Estructura (src/main/webapp/app/):

```

app.tsx                # Punto de entrada y lógica de layout dual
routes.tsx             # Rutas (landing + dashboard + auth)
dashboard/             # Panel administrativo custom
config/                # Redux Toolkit, dayjs, interceptores Axios
entities/              # CRUDs generados por JHipster (NO MODIFICAR)
modules/               # Módulos JHipster (NO MODIFICAR)
shared/                # Componentes/utilidades JHipster (NO MODIFICAR)
landing/               # Tienda pública y panel de cliente
├─ components/         # Componentes reutilizables
├─ pages/              # Páginas de la tienda y /cuenta
├─ hooks/              # useCart, useCatalog, useCuentaActual
├─ services/           # Llamadas a API (checkout, etc.)
└─ utils/              # Formateo, errores, constantes

```

└─ context/

CartContext

Características del frontend

- Layout dual: app.tsx decide entre LandingLayout (tienda), AccountLayout (/cuenta) y layout clásico JHipster (admin/CRUD).
- Enrutamiento con Outlet: LandingLayout carga catálogo y carrito una vez y renderiza <Outlet />.
- Estado global: Redux Toolkit (auth, perfil, entidades). Carrito anónimo en localStorage (knstore-cart).
- Responsividad: obligatoria desde 360 px hasta 1920 px.

3.3 Modelo de dominio destacado

Entidades principales

- Catálogo: Categoría, Subcategoría, Marca, Producto, ProductoPrecio, ProductoInventario, ProductoImagen, EtiquetaProducto, CategoricalVA.
- Clientes: User (JHipster) + Cuenta (perfil extendido).
- Pedidos: Pedido, ItemPedido, Pago, Envío, Factura.
- Carrito: Carrito, ItemCarrito.
- Auditoría: HistorialEstado.
- Secuencias: Secuencia (consecutivos diarios atómicos).

Relaciones clave

- Cuenta 1:1 User
- Carrito 1:1 Cuenta
- Pedido 1:1 Dirección
- Pedido 1:1 Envío
- Producto 1:1 ProductoPrecio / ProductoInventario
- Producto 1:N ProductoImagen

4. Roles y seguridad

4.1 Roles definidos

Rol	Descripción
ROLE_ADMIN	Acceso total: administración, usuarios, configuraciones críticas, auditoría.
ROLE_MANAGER	Acceso operativo: inventario, servicios, distribución, catálogo.
ROLE_CLIENTE	Cliente de la tienda: catálogo, carrito, pedidos propios, perfil (/cuenta).
ROLE_USER	Usuario genérico; sin acceso a entidades de negocio protegidas.

4.2 Permisos por ruta (frontend)

Ruta	Acceso
/, /categorias, /productos, /buscar	Público
/carrito	Público (local/anónimo)
/checkout	Autenticado (ADMIN, MANAGER, CLIENTE, USER)
/cuenta/*	Autenticado (panel propio del cliente)
/admin/*	ADMIN, MANAGER
/account/settings, /account/password	ADMIN, MANAGER
/login, /account/register, /account/activate, /account/reset/*	Público

4.3 Ownership de recursos

Para Cuenta, Direccion, Carrito, Pedido, ItemCarrito, ItemPedido, Pago, Envio, Factura:

- ADMIN / MANAGER: acceso total a recursos de cualquier cliente.
- CLIENTE: solo lectura/escritura/borrado de sus propios recursos.
- USER: acceso denegado a endpoints protegidos.

4.4 Consideraciones de seguridad por diseño

- POST /api/pagos/callback es server-to-server: solo ADMIN / MANAGER.
- El cliente paga vía POST /api/pagos/iniciar; la pasarela simulada auto-aprueba en el servidor.
- POST /api/pedidos/{id}/cancelar: cancelación con máquina de estados + restauración de stock.
- Los PATCH/PUT de Pedido / Pago / Carrito / Factura / Envio / ItemPedido son solo ADMIN / MANAGER (anti mass-assignment).
- Precios siempre server-side: checkout e ItemCarrito ignoran cualquier precioUnitario enviado por el cliente; se resuelven desde Producto.precio.precioVenta en base de datos.

5. Requisitos previos

5.1 Software necesario

- Java JDK 21
- Maven 3.2.5+
- Node.js 24.16.0 LTS
- Docker + Docker Compose
- Git

5.2 Cuentas / servicios externos (opcionales)

- Servidor SMTP para notificaciones transaccionales.
- Pasarela de pagos real (el proyecto incluye pasarela simulada configurable).

6. Instalación y configuración

6.1 Clonar el repositorio

```
git clone https://github.com/joseph12n/knstore.git
cd knstore
```

6.2 Instalar dependencias del frontend

```
./npmw install
```

6.3 Archivos de configuración principales

Archivo	Propósito
<code>pom.xml</code>	Dependencias y plugins Maven.
<code>package.json</code>	Scripts y dependencias Node.
<code>src/main/resources/config/application.yml</code>	Configuración central de Spring Boot.
<code>src/main/resources/config/application-dev.yml</code>	Perfil de desarrollo.
<code>src/main/resources/config/application-prod.yml</code>	Perfil de producción.
<code>.../config/SecurityConfiguration.java</code>	Reglas de seguridad HTTP.

7. Base de datos

7.1 Modos de ejecución de MongoDB

Archivo	Modo	Uso
<code>src/main/docker/mongodb.yml</code>	Standalone, puerto 27018	EC2 / DNS (app.knstore.duckdns.org); compatible sin transacciones.
<code>src/main/docker/mongodb-replicaset.yml</code>	Replica set rs0 de un nodo, puerto 27018	Desarrollo local con transacciones reales.

7.2 Levantar MongoDB en desarrollo (recomendado)

```
docker compose -f src/main/docker/mongodb-replicaset.yml up -d --wait
```

Verificación:

```
docker exec knstore-mongodb-1 mongosh --quiet mongodb://localhost:27017/admin --eval  
'db.hello().isWritablePrimary'
```

Debe devolver true.

7.3 Seed de catálogo demo

En desarrollo, el catálogo demo se carga automáticamente si está habilitado:

```
knstore:  
  seed:  
    catalog: true
```

La migración es idempotente y no afecta producción.

7.4 Índices

El proyecto crea índices mediante migraciones Mongock:

- ProductoIndexesMigration: índices de texto y por campos de catálogo.
- CuentaIndexesMigration: índices para consultas de cuenta.
- SecuenciasIndexMigration: índice único para consecutivos diarios.
- UserUniqueIndexesMigration: índices únicos de usuario.

7.5 Transacciones

El checkout atómico y la concurrencia de stock usan transacciones multi-documento de MongoDB. Requieren un replica set. En entornos standalone (como la EC2 de producción configurada) las transacciones no están disponibles; el flujo debe validarse para ese despliegue.

8. API REST

8.1 Convenciones

- Base: /api/**
- Respuestas en JSON.
- Autenticación: JWT en header Authorization: Bearer <token>.
- Paginación: parámetros page, size, sort.

8.2 Endpoints principales por dominio

Dominio	Endpoints representativos
Autenticación	/api/authenticate, /api/account/**
Usuarios	/api/admin/users/**, /api/users/**
Catálogo	/api/categorias, /api/subcategorias, /api/marcas, /api/productos, /api/productos/por-ids
Cuenta cliente	/api/cuentas, /api/cuentas/mia
Direcciones	/api/direcciones, /api/direcciones/{id}/predeterminada
Carrito	/api/carritos, /api/item-carritos
Checkout	/api/checkout/preview, /api/checkout
Pedidos	/api/pedidos, /api/pedidos/{id}/cancelar
Pagos	/api/pagos/iniciar, /api/pagos/callback
Envíos	/api/envios, /api/envios/pendientes, /api/envios/{id}/tracking
Facturas	/api/facturas, /api/facturas/{id}/pdf
Historial	/api/historial-estados
Management	/management/prometheus (protegido)

8.3 Checkout

1. **Preview:** **POST** `/api/checkout/preview` — calcula totales, envío (gratis $\geq$ \$150.000) e impuestos sin crear pedido.
2. **Checkout:** **POST** `/api/checkout` — crea pedido, items, pago y envío de forma atómica.
3. **Pago:** **POST** `/api/pagos/iniciar` — inicia el pago; con pasarela simulada se auto-aprueba en el servidor.

9. Frontend

9.1 Inicio del entorno de desarrollo

Backend:

```
./npmw run backend:start # Spring Boot en puerto 8080
```

Frontend:

```
./npmw run start # Webpack dev server en puerto 9000
```

9.2 Build de producción

```
./npmw run webapp:prod # Build optimizado para producción
```

9.3 Consejos de desarrollo

- No modificar código autogenerado por JHipster en `entities/`, `modules/` ni `shared/`.
- El código personalizado de la tienda pública y panel de cliente va en `landing/` y `dashboard/`.
- Preferir hooks `useCart` y `useCatalog` en lugar de repetir lógica de fetching.
- Mantener responsividad; probar desde 360 px.

10. Testing

10.1 Backend

```
./mvnw compile                # Compilar
./mvnw -q -DskipITs=false test # Tests unitarios e integración
./mvnw -q -Dtest=ResourceAccessServiceTest test # Test de ownership
./mvnw -Pprod clean verify    # JAR de producción + tests
./mvnw -Pprod,war clean verify # WAR de producción
```

Los tests de integración usan Testcontainers para levantar un replica set de MongoDB.

10.2 Frontend

```
./npmw test          # Tests con Vitest
./npmw run lint       # Linter
npx tsc --noEmit      # Chequeo de tipos TypeScript
```

10.3 JMeter

El repositorio incluye planes de pruebas funcionales y de estrés:

- `docs/knstore_test_plan.jmx` — pruebas funcionales y de seguridad.
- `docs/knstore_stress_plan.jmx` — escenarios de carga ES-01...ES-07.
- `docs/plan_jmeter.md` — matriz de casos y resultados.
- `docs/jmeter/generar_informe.sh` — pipeline para generar informes HTML/PDF desde `resultados.jtl`.

11. Despliegue

11.1 Docker local completo

```
docker compose -f src/main/docker/services.yml up # MongoDB + auxiliares
docker compose -f src/main/docker/app.yml up      # Aplicación completa
```

11.2 Imagen Docker con Jib

```
npm run java:docker
```

11.3 Empaquetado JAR

```
./mvnw -Pprod clean verify
```

El JAR ejecutable se genera en target/.

11.4 Consideraciones de producción

- Configurar JWT secret y credenciales de MongoDB como variables de entorno.
- Configurar servidor SMTP para notificaciones.
- Revisar CORS en application-prod.yml.
- Proteger /management/prometheus.
- Evaluar si el despliegue objetivo requiere transacciones MongoDB; de ser así, usar un replica set.

12. Utilidades y servicios clave

12.1 Servicios destacados del backend

Clase	Responsabilidad
<code>CheckoutService</code>	Lógica atómica de checkout.
<code>PagoService</code> / <code>SimulatedPaymentGateway</code>	Procesamiento de pagos y pasarela simulada.
<code>FacturaService</code> / <code>FacturaPdfService</code>	Facturación, consecutivos y generación de PDF con QR.
<code>EnvioService</code>	Gestión de envíos, tracking y devoluciones.
<code>PedidoService</code>	Máquina de estados de pedidos y cancelaciones.
<code>ResourceAccessService</code>	Validación de ownership de recursos.
<code>SecuenciaService</code>	Consecutivos diarios atómicos (colección secuencias).
<code>MoneyUtils</code>	Normalización de valores monetarios a 2 decimales.
<code>MongoIdUtils</code>	Conversión String↔ObjectId para consultas batch sobre @DBRef.

12.2 Servicios destacados del frontend

Archivo	Responsabilidad
<code>CartContext.tsx</code>	Estado global del carrito (local + backend).

Archivo	Responsabilidad
<code>useCatalog.ts</code>	Fetching y filtros del catálogo.
<code>useCuentaActual.ts</code>	Carga de la cuenta del cliente autenticado.
<code>checkout.service.ts</code>	Llamadas al checkout y pago.
<code>apiError.ts</code>	Manejo tipado de errores de Axios.

13. Convenciones de desarrollo

13.1 Commits (Conventional Commits)

```
<tipo><alcance>: <descripción corta en minúsculas y en español>
```

Tipos: feat, fix, chore, refactor, docs, style, test.

Alcances: backend, frontend, config, webpack, vite, docs.

Ejemplos:

```
feat(backend): permitir a CLIENTE crear su propia Cuenta con validación
chore(config): configurar CORS y URL base para producción
fix(webpack): usar URL relativa para API en producción
```

13.2 Flujo de trabajo Git

- Trabajar en ramas dedicadas (feature/modulo-inventario, bugfix/error-login).
- Commits atómicos.
- Validación local antes de push: backend compila, TypeScript sin errores.
- Pull Requests con revisión de al menos un compañero.

14. Archivos clave de referencia

Archivo	Descripción
<code>README.md</code>	Presentación general del proyecto.
<code>AGENTS.md</code>	Contexto del proyecto para agentes de IA.
<code>CONTRIBUTING.md</code>	Guía de contribución y convenciones de commits.
<code>docs/REQUERIMIENTOS_PENDIENTES.md</code>	Requerimientos pendientes y deuda técnica conocida.
<code>docs/ESTADO_SESION.md</code>	Handoff con estado git/docker/quirks del trabajo activo.

Archivo	Descripción
<code>docs/MONGO-SCHEMA-VALIDATION.md</code>	Validación y esquema de MongoDB.
<code>knstore.jdl</code>	Definición del dominio JHipster.
<code>.yo-rc.json</code>	Configuración del generador.

15. Solución de problemas comunes

15.1 El checkout falla por transacciones

Asegúrate de estar corriendo MongoDB como replica set:

```
docker compose -f src/main/docker/mongodb-replicaset.yml up -d --wait
```

15.2 El frontend no encuentra el backend

Verifica la URL base de la API en el interceptor de Axios y la configuración de CORS.

15.4 Seed de catálogo no aparece

Verifica que `knstore.seed.catalog=true` en `application-dev.yml` y que las migraciones de Mongock se ejecutaron sin errores.